

I Tiny Text-Conditioned Avatar Diffusion from Scratch

1 Task Overview

In this assignment, you will build a small text-conditioned diffusion model that generates cartoon avatars from short structured descriptions. The goal is not to call a ready-made text-to-image system: you must implement and train the main model from scratch on a small image dataset. The final result should be both technically inspectable and easy to demonstrate. A user should be able to enter a prompt such as “a blue cartoon avatar with round eyes and exaggerated proportions”, choose a seed, and inspect the generated 32x32 or 64x64 image.



Figure 1: Illustrative target space for the assignment: a small set of cartoon avatars with different text-controllable attributes such as hair, eyes, accessories, proportions, and color accents.

The research question is:

Can a very small diffusion model, whose denoiser and text encoder are both trained from scratch, generalize to combinations of avatar attributes that were not observed together during training?

2 Objective

The main goal is to design, implement, train, and evaluate a complete pixel-space DDPM pipeline. Your solution must include:

Sub-Objectives

- a reproducible image and caption preprocessing pipeline;
- a text tokenizer and text encoder trained from scratch;
- a compact U-Net denoiser trained from scratch;



- a text-conditioning mechanism;
- a noise schedule, forward noising process, reverse sampling loop, and checkpointing;
- evaluation on both ordinary and compositional out-of-distribution prompts.

3 Dataset

The recommended dataset is the Google Cartoon Set, which contains labelled 2D cartoon avatars in 10k and 100k versions. See the official page: [Google Cartoon Set](#).

The dataset attributes should be converted into deterministic, multi-attribute captions. For example:

“a boy with blue colors, round eyes, and exaggerated proportions”

The caption vocabulary and tokenizer must be derived from the training split only. A 10k subset is an acceptable fallback; the 100k version is recommended for the extended configuration.

4 Data Preprocessing

All preprocessing steps should be clearly documented and justified in your report. Resize the avatars to the selected resolution, normalize the image values consistently, and generate deterministic multi-attribute captions from the dataset metadata. The tokenizer vocabulary must be constructed from the training split only.

Do not use validation or test captions, normalization statistics, or attribute frequencies when constructing the training representation. Save the split definition and preprocessing configuration so that the complete pipeline can be reproduced.

Compositional Split

The main split must be defined over attribute combinations rather than only over individual images. Hold out a controlled set of combinations, for example a particular color–artwork or color–proportion combination, and ensure that the corresponding combination is absent from training.

You must report:

- which individual attributes appear in training;
- which combinations are held out;
- the number of train, validation, ordinary test, and compositional-OOD examples;
- how the caption vocabulary is constructed without using validation or test text.

Mandatory From-Scratch Constraints

The following components are forbidden in the main solution:

- Stable Diffusion, Tiny-SD, SDXL, Flux, or any other pretrained diffusion checkpoint;
- CLIP, T5, BERT, or any pretrained text encoder used for conditioning;



- a pretrained VAE or latent diffusion pipeline;
- a ready-made Diffusers training pipeline used as a black box;
- pretrained image or text embeddings as the main representation.

You may use PyTorch and ordinary tensor/optimization utilities. Standard layers such as convolutions, normalization, embeddings, and attention are allowed, but the U-Net forward pass, text encoder, conditioning path, diffusion objective, noise schedule, and reverse sampling loop must be implemented and documented by you.

5 Model Architecture

The recommended minimum model is a pixel-space DDPM with a compact U-Net. A reasonable envelope is:

- base channels: 64-128;
- two or three spatial resolutions;
- residual blocks with time embeddings;
- a 2-4 layer Transformer text encoder trained from scratch;
- text hidden size: 64-128;
- conditioning injected through cross-attention or other techniques.

6 Hints

You can:

- begin with a 32x32 smoke test and verify that the model can overfit a very small subset before scaling up;
- cache the deterministic captions, vocabulary, and split assignments to make every experiment reproducible;
- inspect generations across several seeds rather than selecting only the most attractive sample;
- keep the ablations small and focused so that the T4 budget is spent on analysis rather than on a broad hyperparameter sweep.

7 Training and Evaluation

Train the model by sampling a timestep, adding noise to a clean image, and predicting the corresponding noise target or velocity target. Use a fixed random seed for the main run and save enough information to resume training.

The minimum required experiments are:

- unconditional neural baseline;



- conditional model with the selected conditioning mechanism;

Do not spend the budget on a broad hyperparameter sweep. A small number of justified experiments and a strong analysis are preferred.

Mandatory metrics: Your evaluation must include both quality and conditioning metrics:

- image quality metric such as FID or KID, with all implementation details documented;
- diversity across seeds for the same prompt;
- parameter count, sampling time, and memory usage.

8 Deliverables

- A well-commented Python code implementation of the data preprocessing and model tuning pipeline.
- A report documenting the detailed approach, including data preprocessing, the model training, and evaluation results.
- Analysis and discussion of the model's performance, challenges faced, and recommendations for improvement.
- A presentation that will be discussed at the time of the Oral Exam.

Notes

You are encouraged to utilize existing deep learning libraries such as TensorFlow or PyTorch for implementing the model. Make sure to properly document your code and provide clear explanations in the report to demonstrate your understanding of the concepts and techniques used.